ASSIGNMENT NO. 1

UNIT III

Inheritance, Polymorphism, Exception Handling and Multithreading

Q1. Explain superclass and subclass in Java. Write a program to demonstrate different types of inheritance supported by Java.

Superclass vs. Subclass: A *superclass* (parent class) is the class whose properties and methods are inherited. A *subclass* (child class) is the class that derives features from the superclass using the `extends` keyword. Subclasses can reuse superclass code and add unique behavior.

Supported Inheritance Types: Java supports **Single**, **Multilevel**, and **Hierarchical** inheritance. Java does *not* support multiple inheritance using classes to avoid ambiguity (Diamond Problem).

```java
// 1. Single Inheritance: Animal -> Dog
class Animal {
    void eat() { System.out.println("Eating..."); }
}

class Dog extends Animal {
    void bark() { System.out.println("Barking..."); }
}

// 2. Multilevel Inheritance: Animal -> Dog -> Puppy
class Puppy extends Dog {
    void weep() { System.out.println("Weeping..."); }
}

// 3. Hierarchical Inheritance: Animal -> Cat (Dog also extends Animal)
class Cat extends Animal {
    void meow() { System.out.println("Meowing..."); }
}

public class InheritanceDemo {
    public static void main(String[] args) {
        Puppy p = new Puppy();
        p.eat();  // Inherited from Animal
        p.bark(); // Inherited from Dog
        p.weep(); // Own method

        Cat c = new Cat();
        c.eat();  // Inherited from Animal
        c.meow(); // Own method
    }
}
```

Q2. Write a Java program to demonstrate constructor execution in inheritance using super() and explain the order of constructor invocation.

Order of Constructor Invocation: When an instance of a subclass is created, constructors are executed top-down, starting from the base superclass down to the derived subclass. The `super()` call must be the very first statement inside a subclass constructor to explicitly pass arguments to the parent constructor.

```
class Parent {
    Parent(String msg) {
        System.out.println("1. Parent Constructor: " + msg);
    }
}

class Child extends Parent {
    Child() {
        super("Hello from Child"); // Invokes Parent constructor first
        System.out.println("2. Child Constructor Executed");
    }
}

public class ConstructorOrderDemo {
    public static void main(String[] args) {
        Child obj = new Child();
    }
}
```

Q3. Explain method overriding and dynamic method binding in Java with a suitable program.

Method Overriding: Declaring a method in a subclass that already exists in the parent class with the exact same name, return type, and parameters.

Dynamic Method Binding (Run-time Polymorphism): The decision of which overridden method to call is made dynamically at runtime based on the actual object being referenced, rather than the reference type variable.

```
class  {
     () {  ; }
}

    {
     () {  ; }
}

    {
     () {  ; }
}

  {
     ([] args) {
         myBank;

        myBank =  ();
        System.out.( + myBank.() + );

        myBank =  ();
        System.out.( + myBank.() + );
    }
}Bankdoublee getInterestRatereturn0.0classSBIextendsBankdoublegetInterestRatereturn6.5//
Overriddenclass HDFCextendsBankdoublegetInterestRatereturn7.2// Overriddenpublic
classDynamicBindingDemopublic static voidmainStringBank// Superclass Referencenew SBI//
Pointing to SBI Objectprintln"SBI Interest Rate: "getInterestRate"%"newHDFC// Pointing
to HDFC Objectprintln"HDFC Interest Rate: "getInterestRate"%"
```

Q4. What are abstract classes and abstract methods? Write a Java program demonstrating their use.

An **abstract class** cannot be instantiated directly and is declared with the `abstract` keyword. An **abstract method** has no method body (implementation) in the parent class and *must* be overridden by concrete subclasses.

```
abstract class  {
     ();

     () {
        System.out.();
    }
}

    {
     () {
        System.out.();
    }
}

  {
     ([] args) {
         s =  ();
        s.();
        s.();
    }
}Shapeabstract voiddraw// Abstract method (no body)voiddisplay// Concrete
methodprintln"Drawing Shape Architecture..."classRectangleextendsShapevoiddraw// Must
implement abstract methodprintln"Drawing a Rectangle"public classAbstractDemopublic
static voidmainStringShapenewRectangledisplaydraw
```

Q5. Explain final methods and final classes in Java with suitable examples.

The `final` keyword provides immutability and restrictions:

- **Final Method:** Prevents method overriding in child classes.
- **Final Class:** Prevents inheritance entirely (the class cannot be extended).

```java
// Final class cannot be extended
final class SecuritySystem {
    void authenticate() { System.out.println("Authenticating user..."); }
}

class BaseFeature {
    // Final method cannot be overridden
    final void showVersion() {
        System.out.println("Version 1.0.0");
    }
}

public class FinalDemo extends BaseFeature {
    public static void main(String[] args) {
        FinalDemo obj = new FinalDemo();
        obj.showVersion();

        SecuritySystem sec = new SecuritySystem();
        sec.authenticate();
    }
}
```

Q6. Explain the different types of exceptions in Java. Write a program demonstrating try, catch, finally, throw, and throws.

Types of Exceptions:

- **Checked Exceptions:** Checked at compile time (e.g., `IOException`, `SQLException`).
- **Unchecked Exceptions:** Occur at runtime (e.g., `ArithmeticException`, `NullPointerException`).

Keywords: `try` defines code to monitor, `catch` handles exceptions, `finally` executes regardless of exceptions, `throw` explicitly throws an exception, and `throws` declares signatures that may raise exceptions.

```
public class ExceptionDemo {
    // Throws clause declaring exception
    static void checkAge(int age) throws IllegalArgumentException {
        if (age < 18) {
            throw new IllegalArgumentException("Access Denied: Age under 18"); // Throw
statement
        } else {
            System.out.println("Access Granted!");
        }
    }

    public static void main(String[] args) {
        try {
            System.out.println("Testing Age Validation...");
            checkAge(15);
        } catch (IllegalArgumentException e) {
            System.out.println("Caught Exception: " + e.getMessage());
        } finally {
            System.out.println("Finally block execution completed.");
        }
    }
}
```

Q7. Write a Java program to demonstrate multiple catch blocks and nested try statements.

Multiple `catch` blocks allow handling different exception types individually. A `try` block can be placed inside another `try` block (nested try) to handle granular execution errors at multiple levels.

```
public class  {
     ([] args) {
         {
            [] arr = [];

             {
                 res =  / ;
            }  ( e) {
                System.out.();
            }

            arr[] = ;

        }  ( e) {
            System.out.();
        }  ( e) {
            System.out.();
        }
    }
}NestedTryDemopublic static voidmainStringtry// Outer Tryintnew int5try// Inner
Tryint300// Division by ZerocatchArithmeticExceptionprintln"Inner Catch: Division by
zero handled."1050// Array index out of
boundscatchArrayIndexOutOfBoundsExceptionprintln"Outer Catch 1: Array index out of
bounds handled."catchExceptionprintln"Outer Catch 2: General Exception handled."
```

Q8. Explain the Java Thread Model. Write a program to create and execute multiple threads.

Java Thread Model: Built around multithreading where lightweight threads execute tasks concurrently within the same program process. A thread progresses through states: *New, Runnable, Running, Blocked/Waiting,* and *Terminated*. Thread execution relies on CPU time-slicing managed by the Thread Scheduler.

```
class MyTask extends Thread {
    private String taskName;
    MyTask(String name) { this.taskName = name; }

    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println(taskName + " executing step " + i);
            try { Thread.sleep(500); } catch (InterruptedException e) {}
        }
    }
}

public class MultiThreadDemo {
    public static void main(String[] args) {
        MyTask t1 = new MyTask("Thread A");
        MyTask t2 = new MyTask("Thread B");
        t1.start();
        t2.start();
    }
}
```

Q9. Explain different ways of creating a thread in Java with suitable programs.

Threads can be created using two primary approaches:

1. **Extending `Thread` class:** Best when extending thread features directly.
2. **Implementing `Runnable` interface:** Preferred because Java supports single class inheritance—implementing an interface keeps class inheritance available for other base classes.

```
// Method 1: Extending Thread class
class ThreadClassWay extends Thread {
    public void run() { System.out.println("Created via Thread Class Extension"); }
}

// Method 2: Implementing Runnable Interface
class RunnableWay implements Runnable {
    public void run() { System.out.println("Created via Runnable Interface
Implementation"); }
}

public class ThreadCreationDemo {
    public static void main(String[] args) {
        ThreadClassWay t1 = new ThreadClassWay();
        t1.start();

        Thread t2 = new Thread(new RunnableWay());
        t2.start();
    }
}
```

Q10. Write a Java program demonstrating multithreading using the Thread class and Runnable interface.

This program demonstrates concurrent multithreaded processing by instantiating one thread via the `Thread` class and another thread via the `Runnable` interface.

```
class WorkerThread extends Thread {
    public void run() {
        System.out.println("[Thread Class] Worker executing...");
    }
}

class WorkerRunnable implements Runnable {
    public void run() {
        System.out.println("[Runnable Interface] Worker executing...");
    }
}

public class CombinedMultithreadDemo {
    public static void main(String[] args) {
        WorkerThread thread1 = new WorkerThread();
        Thread thread2 = new Thread(new WorkerRunnable());

        thread1.start();
        thread2.start();
    }
}
```